한림대학교
HALLYM UNIVERSITY

오픈소스SW의 이해

MIDTERM PREP

SEMESTER | 2026-2

박성미

중간고사 범위

중간고사

섹션
리눅스 명령어 (+ Penguin Gym 명령어도 시험범위)
클래스 (OOP)
Git 이론 (개념 정확히 이해)
Git기초 명령어 & 상태확인
Git기초 명령어 II
GIt 브랜치 & 충돌 해결
지역-원격저장소
오픈소스SW 역사
오픈소스SW라이선스

- 총 20문제 (추가될 수 있음)
- 객관식/단답형 주관식/서술형

화요일 (10/27)

9:30-10:30

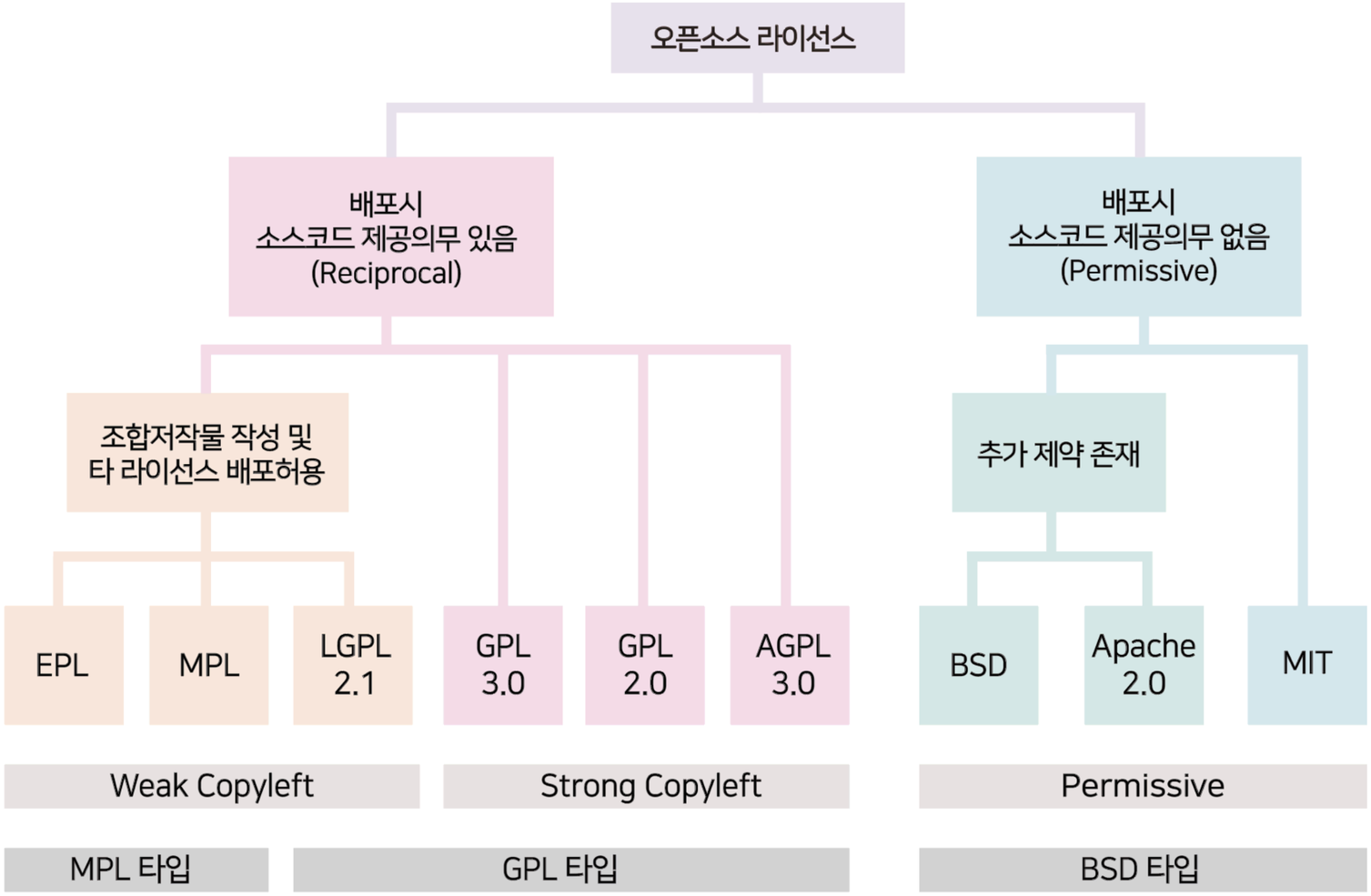
(10분 전 입실 필수)

오픈소스SW 라이선스



오픈소스SW 라이선스

1 오픈소스SW 라이선스의 분류 (복습)



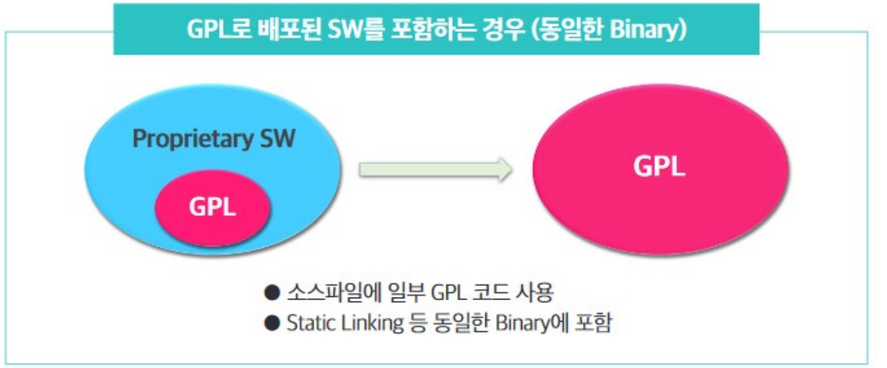
오픈소스SW 라이선스

2 오픈소스SW 라이선스 위반 사례 (문의)

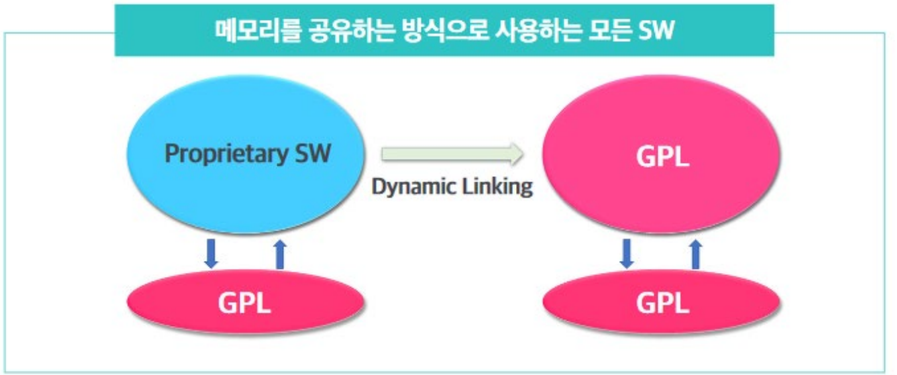
❖ GPL (GNU General Public License)

- 주요 특징
  - 자유 소프트웨어 재단(FSF)에서 만든 강력한 카피레프트 라이선스
  - 소프트웨어의 자유로운 사용, 수정, 공유를 보장

  - **스트롱 카피레프트:** 소스코드 공개 의무, 파생 저작물도 동일한 GPL 적용 필수
  - **배포시 의무사항:**
    - 저작권 고지
    - 라이선스 사본 포함
    - 수정 사항 명시
    - **전체 소스 코드 공개**
      - 저장 공간 등 제약으로 함께 배포할 수 없는 경우 Written Offer 제공 (최소 3년간 제공의무)



※ 바이너리에 GPL 소스코드가 일부라도 포함되어 있으면 바이너리는 GPL이 됨



※ 원 저작자가 특별히 기술한 예외사항이 없는 한 모두 GLP화가 됨

오픈소스SW 라이선스

2 오픈소스SW 라이선스 위반 사례 (문의)

❖ AGPL (Affero General Public License)

- 주요 특징
  - GPL을 기반으로 네트워크 서비스에 특화된 라이선스
  - 서버 측 소프트웨어 소스코드 공개를 요구

  - **스트롱 카피레프트:** 네트워크를 통한 사용자 상호작용 시 소스코드 제공 의무
  - **배포시 의무사항:**
    - 저작권 고지
    - 라이선스 사본 포함
    - **서버에서 실행되는 소프트웨어의 전체 소스코드 공개**
    - 네트워크 사용자에게 소스코드 다운로드 방법 제공

오픈소스SW 라이선스

2 오픈소스SW 라이선스 위반 사례 (문의)



❖ LGPL (Lesser General Public License)

- 주요 특징
  - 자유 소프트웨어 재단(FSF)에서 GPL의 강력한 카피레프트를 완화하기 위해 만든 버전
  - 주로 소프트웨어 라이브러리에 사용

  - **위크 카피레프트:** 라이브러리 **링크** 시 전체 코드 공개 의무 없음, 라이브러리 **자체 수정** 시 소스 코드 공개 필요
  - **배포시 의무사항:**
    - 저작권 고지
    - 라이선스 사본 포함
    - **LGPL 라이브러리 수정 시 해당 부분 소스코드 공개**
    - 사용자가 라이브러리를 수정하고 재링크할 수 있는 방법 제공

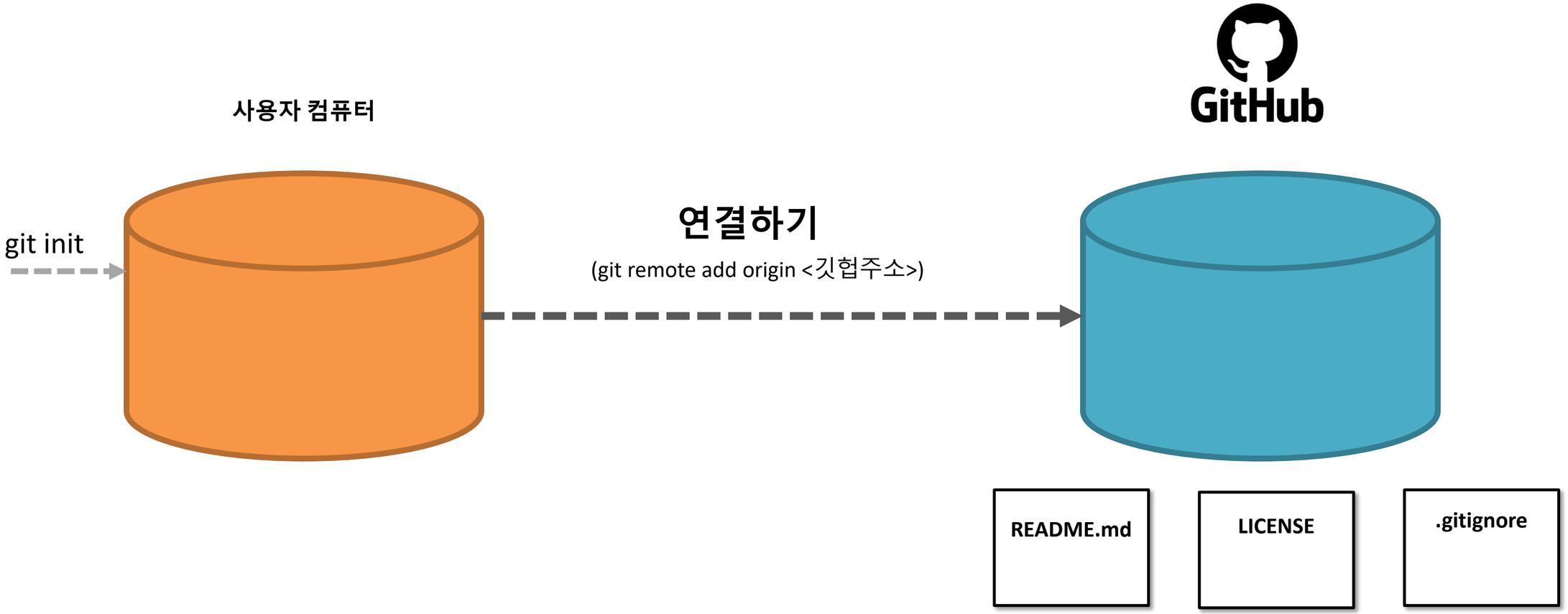




지역- 원격 저장소 연결하기

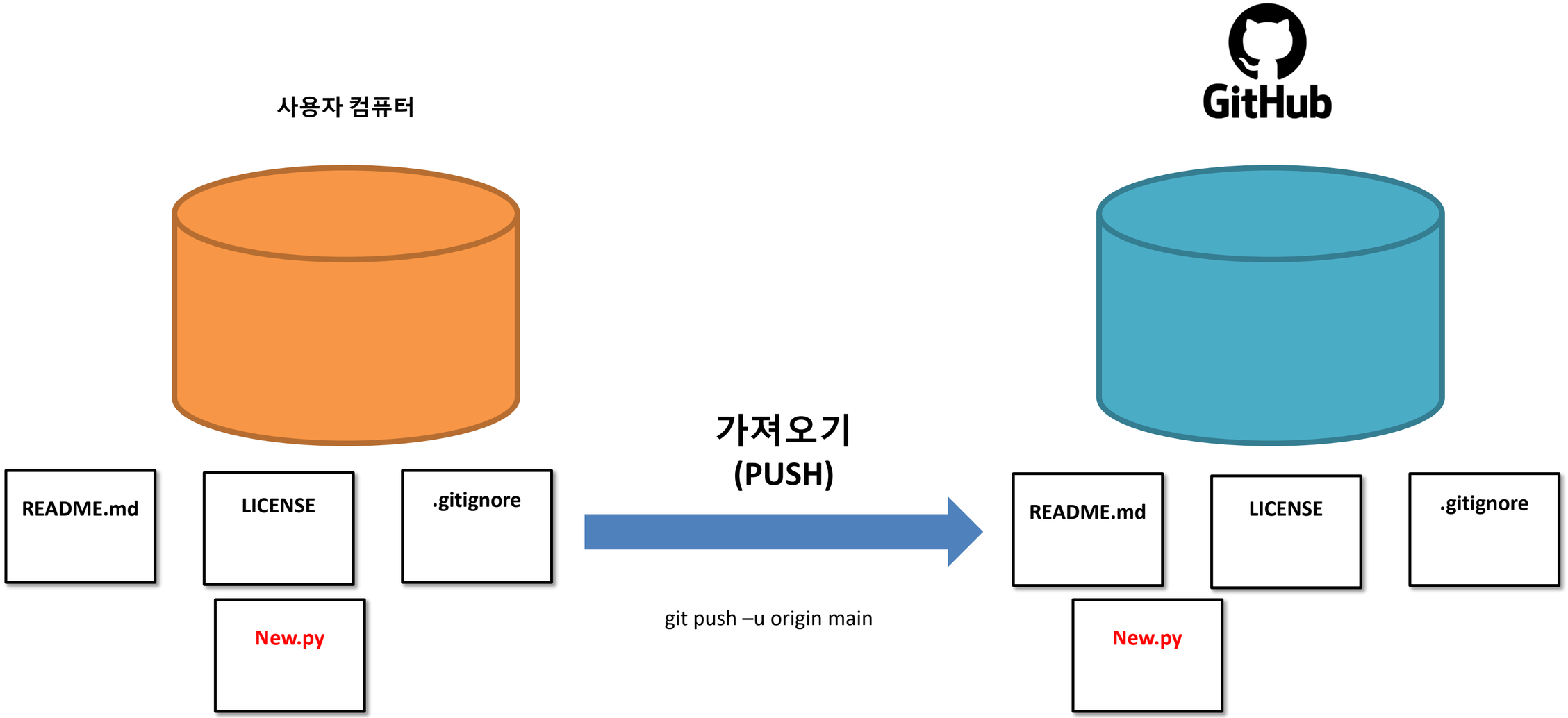
지역-원격 저장소 연결하기

지역-원격 저장소 연결하기 (개념)



지역-원격 저장소 연결하기

지역-원격 저장소 연결하기 (개념)



지역-원격 저장소 연결하기

지역-원격 저장소 연결하기

- 원격 저장소에 연결하기

✓git remote add origin <깃헙주소>

일반적으로 사용하는 원격 저장소 이름

- 연결된 원격 저장소 확인

✓git remote -v

```
PS F:\한림대\오픈소스SW의이해\week7> git remote add origin https://github.com/outspark/git_test.git
```

```
PS F:\한림대\오픈소스SW의이해\week7> git remote -v
origin  https://github.com/outspark/git_test.git (fetch)
origin  https://github.com/outspark/git_test.git (push)
```

지역-원격 저장소 연결하기

커밋 내려받기 (git pull)

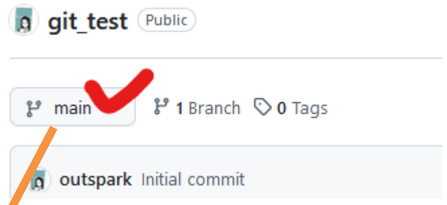
- 브랜치 선택 없이 pull할 경우:

```
PS F:\한림대\오픈소스SW의이해\week7> git pull
There is no tracking information for the current branch.
Please specify which branch you want to merge with.
See git-pull(1) for details.

    git pull <remote> <branch>

If you wish to set tracking information for this branch you can do so with:

    git branch --set-upstream-to=origin/<branch> main
```

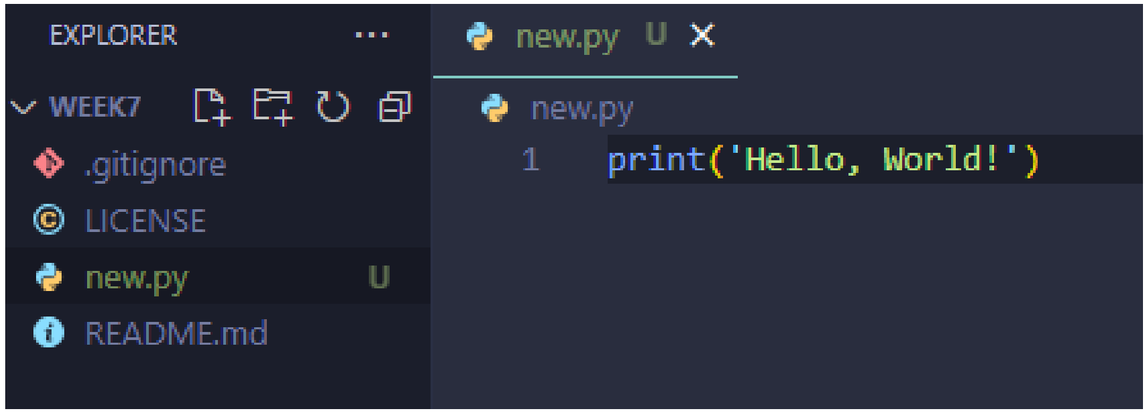



```
PS F:\한림대\오픈소스SW의이해\week7> git pull origin main
From https://github.com/outspark/git_test
 * branch            main       -> FETCH_HEAD
PS F:\한림대\오픈소스SW의이해\week7> git branch
* main
PS F:\한림대\오픈소스SW의이해\week7>
```



지역-원격 저장소 연결하기

커밋 올리기기 (git push)




```
PS F:\한림대\오픈소스SW의이해\week7> git add .
PS F:\한림대\오픈소스SW의이해\week7> git commit -m 'new python'
[main 9fde690] new python
 1 file changed, 1 insertion(+)
 create mode 100644 new.py
```



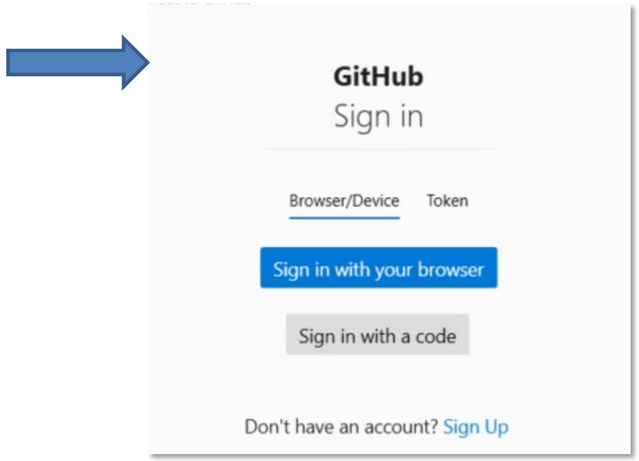
지역-원격 저장소 연결하기

Push할 브랜치 연결해주기

- git push –u origin main 또는
- git push --set-upstream origin main

PUSH하고자 하는 LOCAL 브랜치

설정해준 원격 저장소 이름



인증 요청

```
PS F:\한림대\오픈소스SW의이해\week7> git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 28 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 284 bytes | 284.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/outspark/git_test.git
   3d55574..9fde690  main -> main
branch 'main' set up to track 'origin/main'.
```

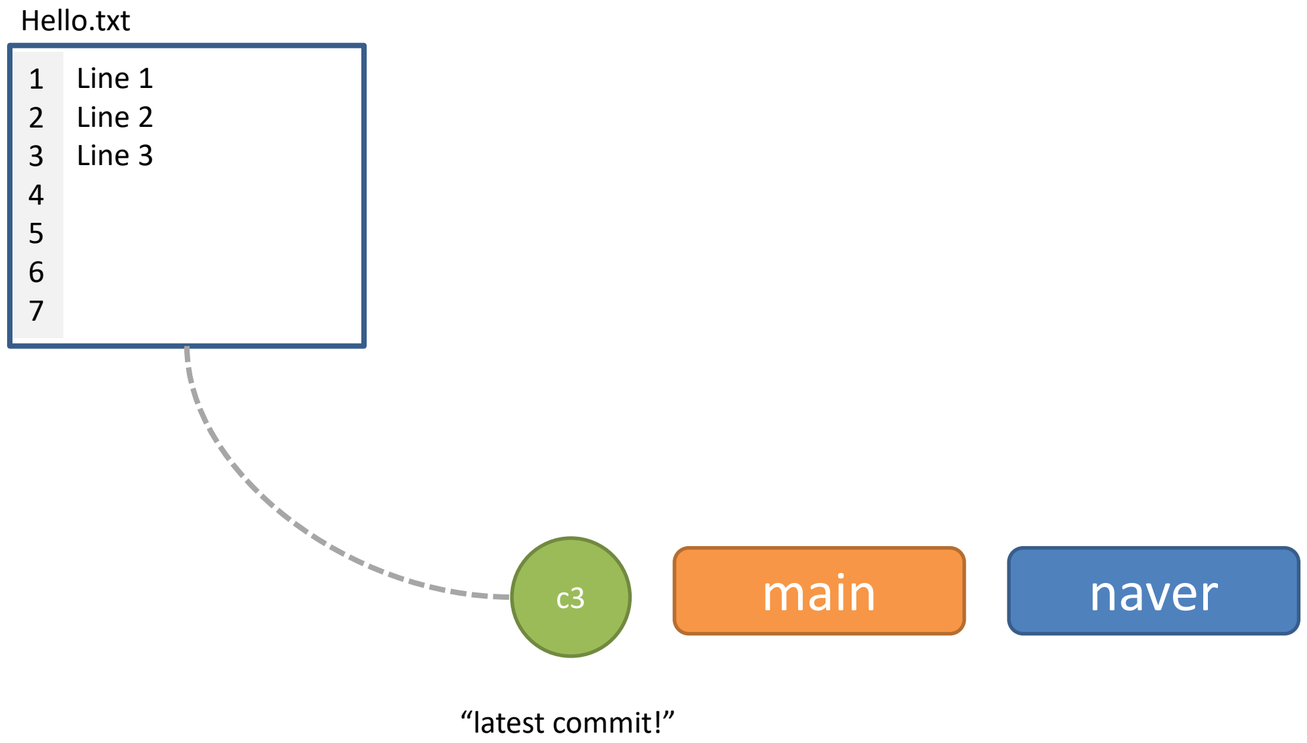


Branch 병합하기



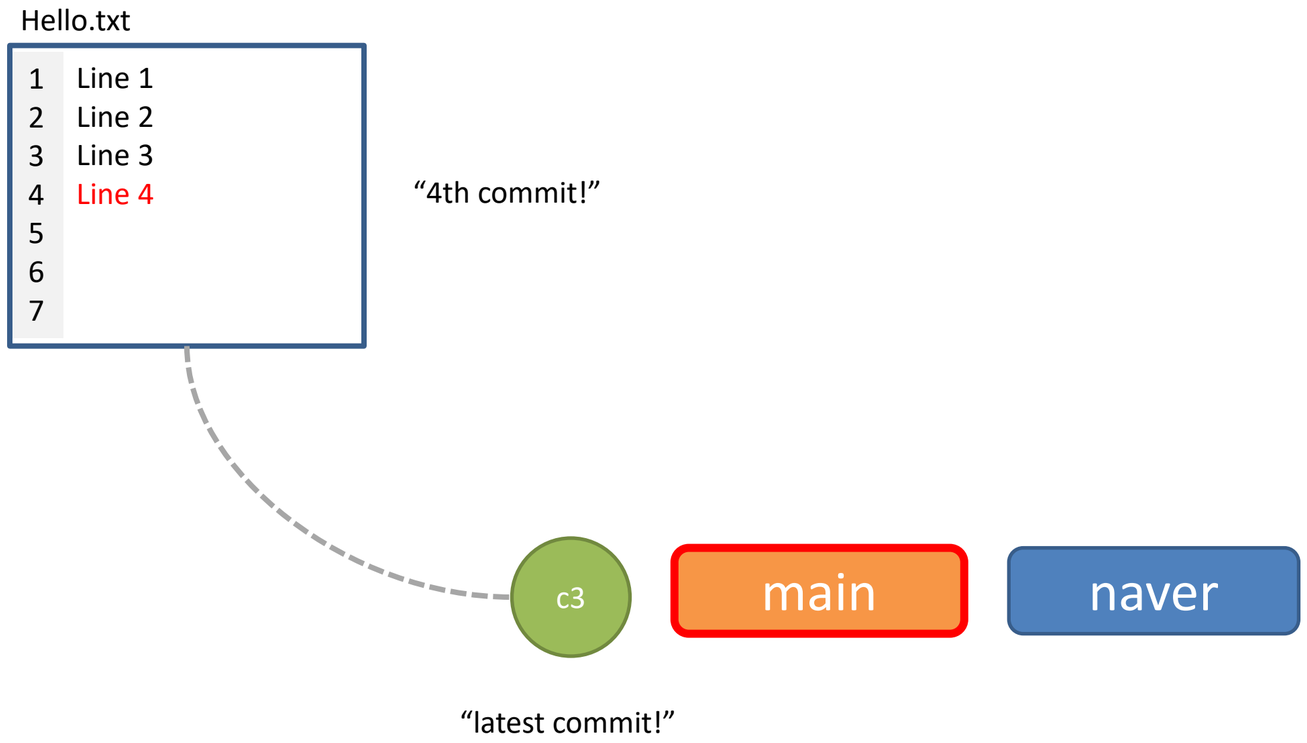
Branch 병합하기

브랜치 병합하기



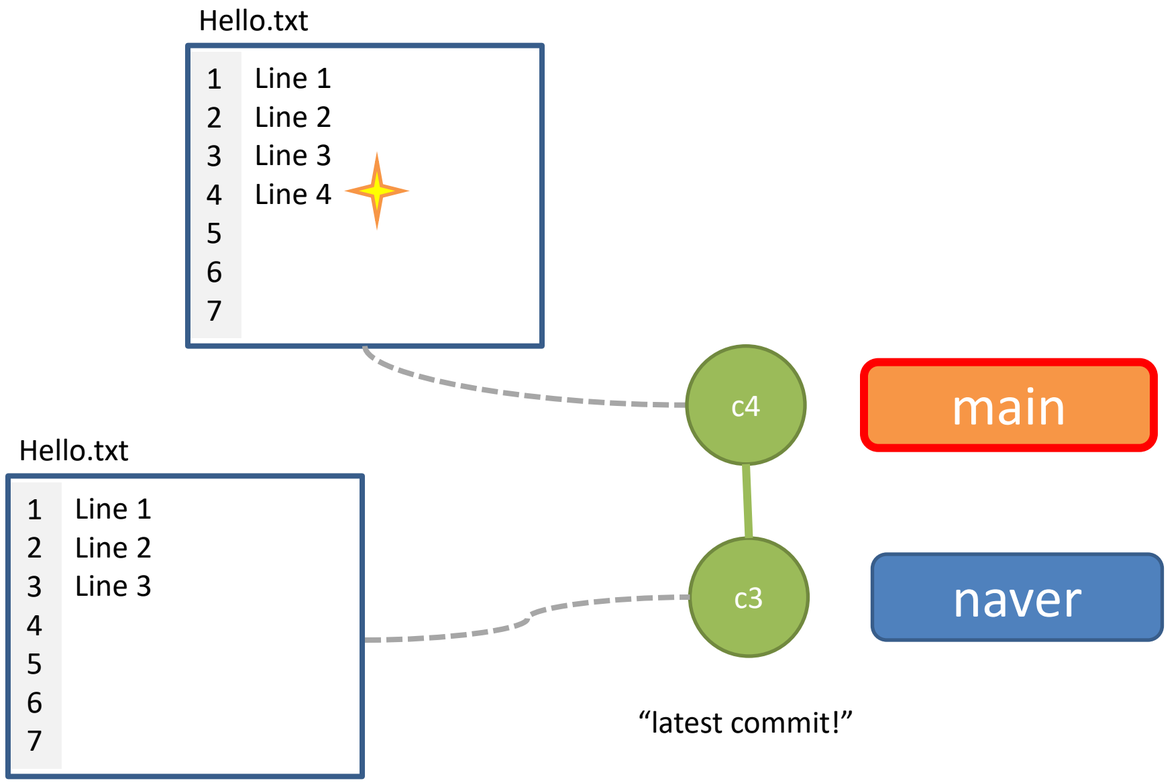
Branch 병합하기

브랜치 병합하기



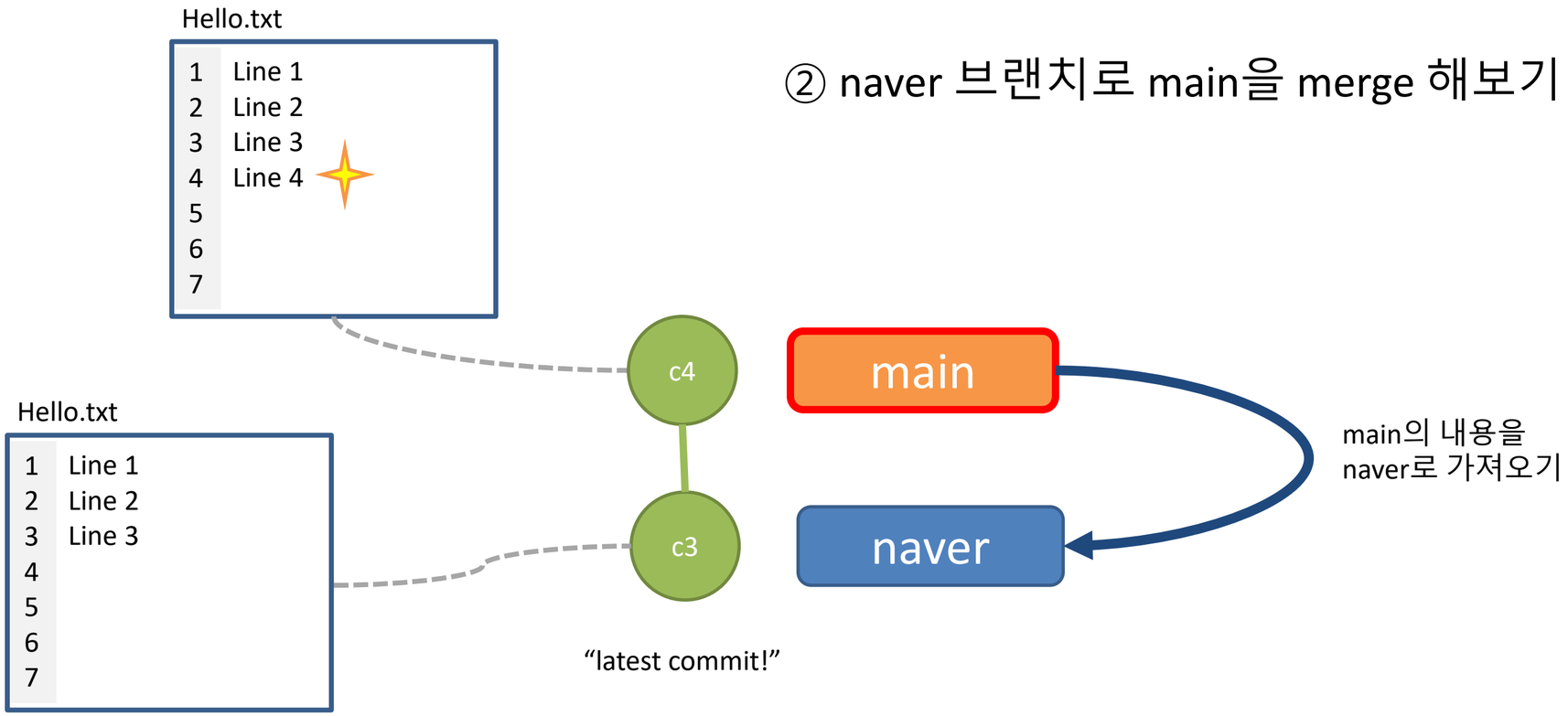
Branch 병합하기

브랜치 병합하기



Branch 병합하기

브랜치 병합하기



Branch 병합하기

Git Branch Merge



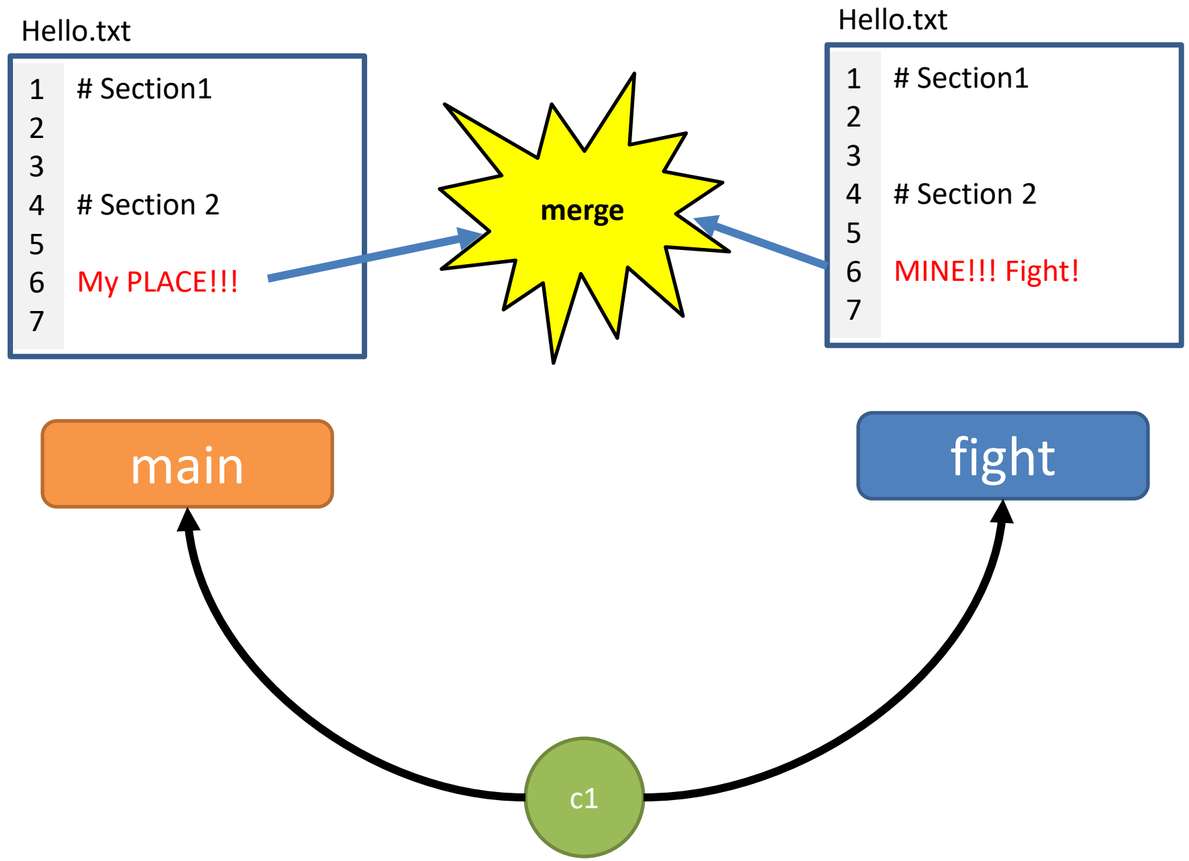
❖ **Fast-Forward (빨리 감기) 병합**

- Branch를 분기한 이후 아무 변화가 없다면 (새로운 커밋x) Git에서 빠른 병합 가능
  - ➢ 포인터만 이동

충돌 해결하기

충돌 해결하기

다른 브랜치에서 같은 부분 수정한 후 병합



충돌 해결하기

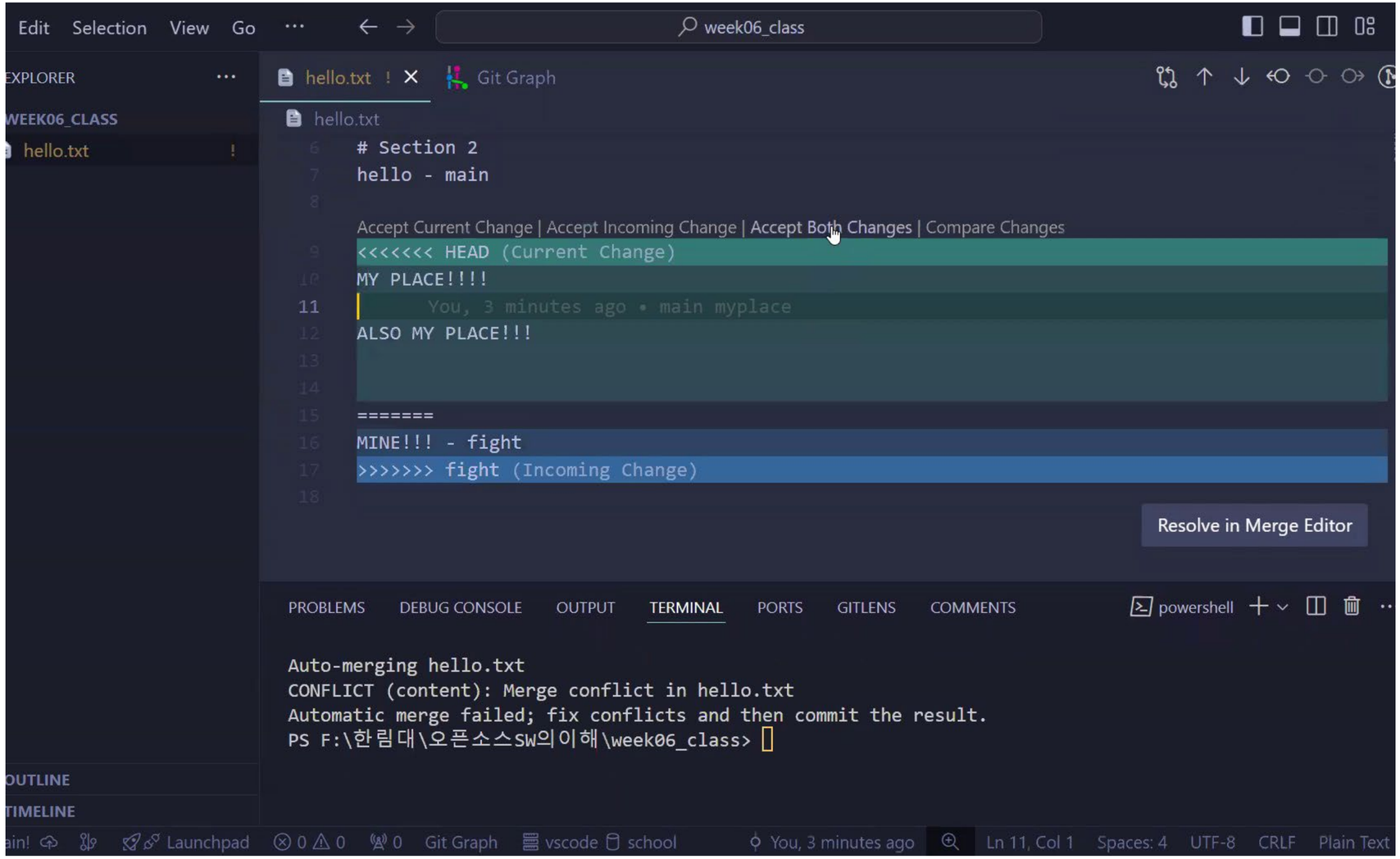
브랜치 병합하기

① `main` 브랜치에서 `fight` 브랜치를 병합 시도

- git merge fight

❖충돌 발생 확인 메시지:

– Automatic merge failed; fix conflicts and then commit the result.

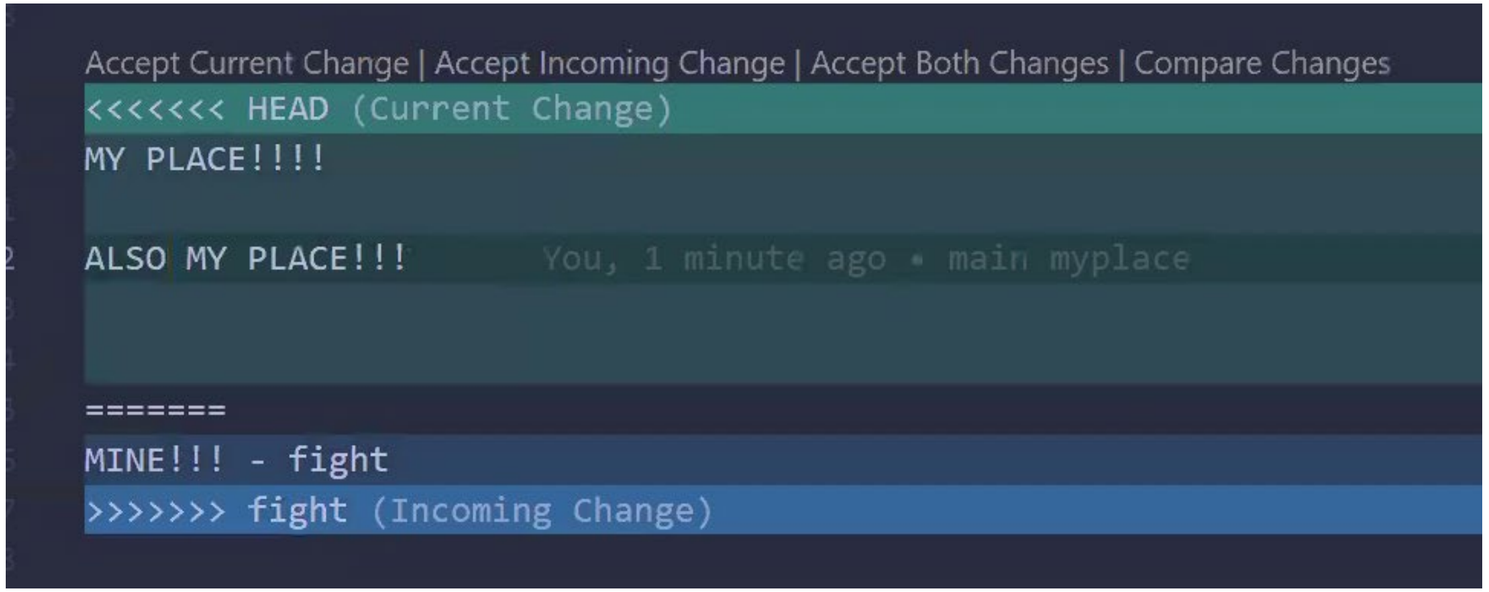


충돌 해결하기

충돌 확인 및 해결

❖ 충돌이 발생한 파일 `hello.txt`를 열면 다음과 같은 표시를 확인 가능:

<<<<<<< HEAD
현재 변경 내용 (main 브랜치)
=======
들어오는 변경 내용 (fight 브랜치)
>>>>>>> fight

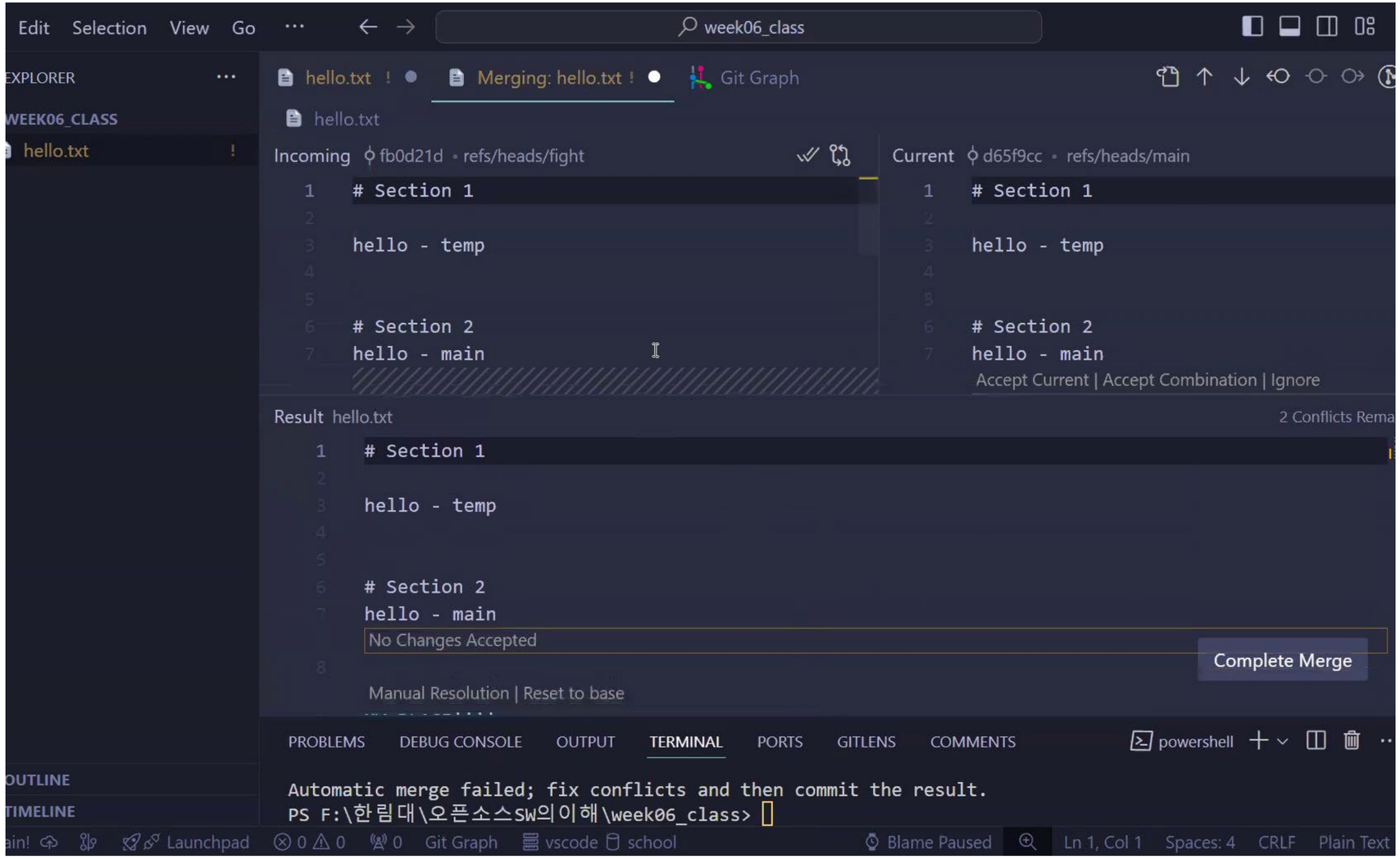


- VS Code는 충돌 부분에 대해
  1) Accept Current Change(현재 변경 사항 수락): : `main` 브랜치의 변경 내용 유지
  2) Accept Incoming Change (들어오는 변경 사항 수락): `fight` 브랜치의 변경 내용 적용
  3) Accept Both Changes,
  4) Compare Changes
  5) Ignore (Resolve Manager) 등의 옵션 제공

충돌 해결하기

충돌 해결하기

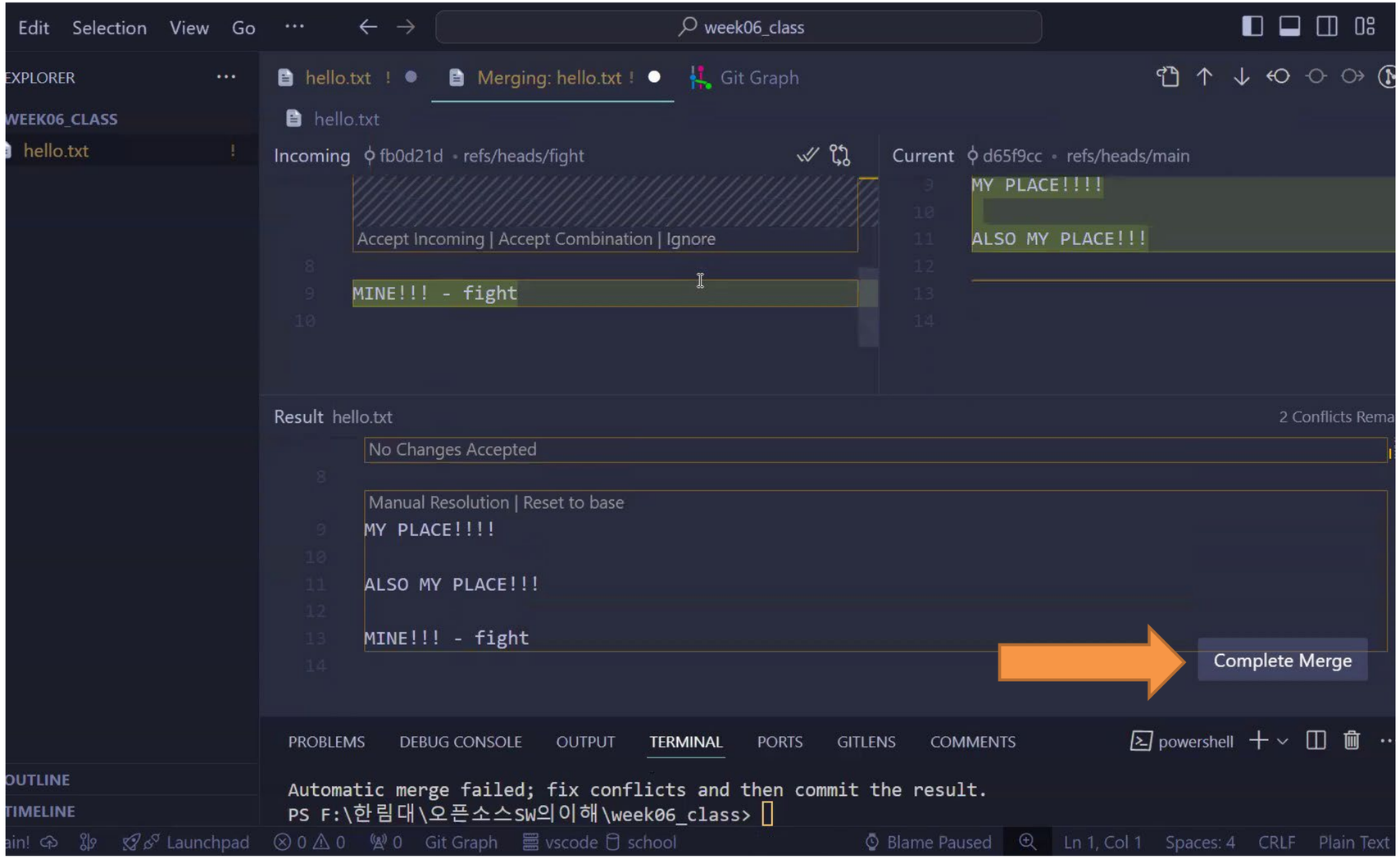
Resolve Manager ➔



충돌 해결하기

병합 완료 및 커밋하기

✓충돌을 모두 해결한 후
✓변경된 파일을 추가하고 커밋



시험 문제 예시

문제 예시

03. 아래 오류 메시지를 보고, 이를 해결하기 위한 Git 명령어를 작성하시오. (2.5점)

#03

```
git status
```

```
fatal: not a git repository (or any of the parent directories): .git
```

문제 예시

02. 에릭 레이먼드(Eric Raymond)는 저서 「The Cathedral and the Bazaar」에서 두 가지 오픈소스 개발 방식을 제시하였다. (1) 성당 모델(Cathedral Model)과 (2) 시장 모델(Bazaar Model)의 개념을 각각 설명하고, 두 모델의 차이점을 ①개발환경, ②릴리스 주기 2가지 측면에서 비교하여 서술하시오.

모델	성당 모델	시장 모델
개념	(정의)	(정의)
개발환경	①-1	①-2
릴리스 주기	②-1	②-2

문제 예시

5 성당과 시장 모델

❖ 성당 모델 vs 시장 모델

구분	성당 모델 (GNU 프로젝트)	시장 모델 (리눅스 커널)
개발 환경	폐쇄적인 개발 환경	개방된 개발 환경
참여자	소수의 개발자들이 소스 코드를 관리하고 외부인에게 제한된 접근 허용	모든 사람이 자유롭게 소스 코드에 접근하고 기여할 수 있음
의사결정 구조	중앙집중식 의사결정 (개발 리더가 모든 결정을 내림)	분산된 의사결정 (개발자들이 자율적으로 문제 해결 및 기여)
릴리스 주기	완성된 버전만 릴리스 릴리스 주기가 길며 **안정적인 버전** 제공	**"자주 릴리스, 자주 배포"** 원칙을 따름 릴리스 주기가 짧고 유동적
릴리스 방식	**완성된 이후에만 코드가 공개** 중간 과정은 외부에 공개되지 않음	코드가 항상 공개되어 있음 개발 중에도 커뮤니티에서 실시간으로 볼 수 있음
예시	GNU Emacs, GCC (폐쇄적 개발과 안정적 릴리스를 강조)	리눅스 커널 (분산형 협업, **"자주 릴리스, 자주 배포"** 원칙 채택)
대표 인물	GNU 프로젝트	리누스 토발즈 (리눅스 창시자, 시장 모델 제시)

오늘은 여기까지.

Thank you for listening.